



Pointers and arrays in C

Lecture overview

- Pointers in C – recap ←
- Pointers as function parameters
- Arrays in C
- Arrays and pointers
- Strings in C
- Pointer arithmetic and array indexing

Pointers

Pointers allow us to access addresses of variables

```
#include <stdio.h>
int main() {
```

```
    int i = 7;
```

```
    int *ip = &i;
```

```
    printf ("    Value of i: %d\n", *ip);
```

```
    printf ("Location of i: %p\n", ip);
```

```
    return 0;
```

```
}
```

← declaring a pointer variable

← dereferencing a pointer

← printing memory address

%p for printing
pointer address

Value of i: 7

Location of i: 0xbfffffff924 ← 8-byte hexadecimal code

Referencing operator - &

Referencing operator always acts on a variable

- If **x** is variable of type **int**, then **&x** is of type **int***
- If **x** is variable of type **double**, then **&x** is of type **double***
- If **x** is variable of type **int***, then **&x** is of type **int****
- ...

The address held by the pointer is the address of the byte with lowest address (among the bytes that hold the variable)

0xbffffe15a45bf924

. . .

0xbffffe15a45bf92b



```
double d = 3.14;
double *p_d = &d;
```

Operator precedence :

- Access and Grouping: `()` (function call), `[]`, `.`, `->`
- Unary: `++`, `--`, `!`, `()` (type casting), `&` (address), `*` (dereference)
- Arithmetic: `*` (multiplication), `/`, `%`; and then `+`, `-`
- Bitwise shifts: `<<`, `>>`
- Relational: `>`, `>=`, `<`, `<=`
- Bitwise `&`; and then `^`; and then bitwise `|`
- Logical `&&`; and then logical `||`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`, etc.

Dereferencing operator - *

Dereferencing operator always acts on an expression of pointer type

- If **exp** is an expression of type `int*`, then `*exp` is of type `int`
- ...

The type of the pointer indicates how many bytes to read and how to interpret them (integer / floating point)

0xbffffe15a45bf924

. . .

0xbffffe15a45bf92b



```
double d = 3.14;
double *p_d = &d;
*p_d+=3;
```

Referencing / dereferencing

Code example:

```
int      i = 3, j = 5,  
        *p = &i, *q = &j, *r;  
  
if(p == &i) printf ("same address\n");
```

- Pointer types can be compared to each other using all relational operators (==, <, > <=, >=)
- The **addresses** are being compared, and not the content!

Referencing / dereferencing

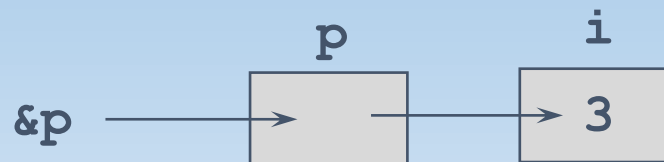
Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)      **/
  
```

- Referencing (&) and dereferencing (*) are inverse operations
 (*&x == x for any variable x)
- The referencing operator (&) can be applied to any type (even a pointer),
 but the dereferencing operator (*) can be applied only to pointer types



Operator precedence :

- Access and Grouping: `()` (function call), `[]`, `.`, `->`
- Unary: `++`, `--`, `!`, `()` (type casting), `&` (address), `*` (dereference)
- Arithmetic: `*` (multiplication), `/`, `%`; and then `+`, `-`
- Bitwise shifts: `<<`, `>>`
- Relational: `>`, `>=`, `<`, `<=`
- Bitwise `&`; and then `^`; and then bitwise `|`
- Logical `&&`; and then logical `||`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`, etc.

Referencing / dereferenc

Code example:

```
int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)     **/
```

- If **p** is of type **int***, then ***p** is of type **int** and can be an operand of an arithmetic operation

Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)     **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;         **/
  
```

- Any expression of pointer type can be dereferenced (even `r+2`)
- If a pointer is dereferenced at the left side of an assignment, then the **value stored at that address is changed**

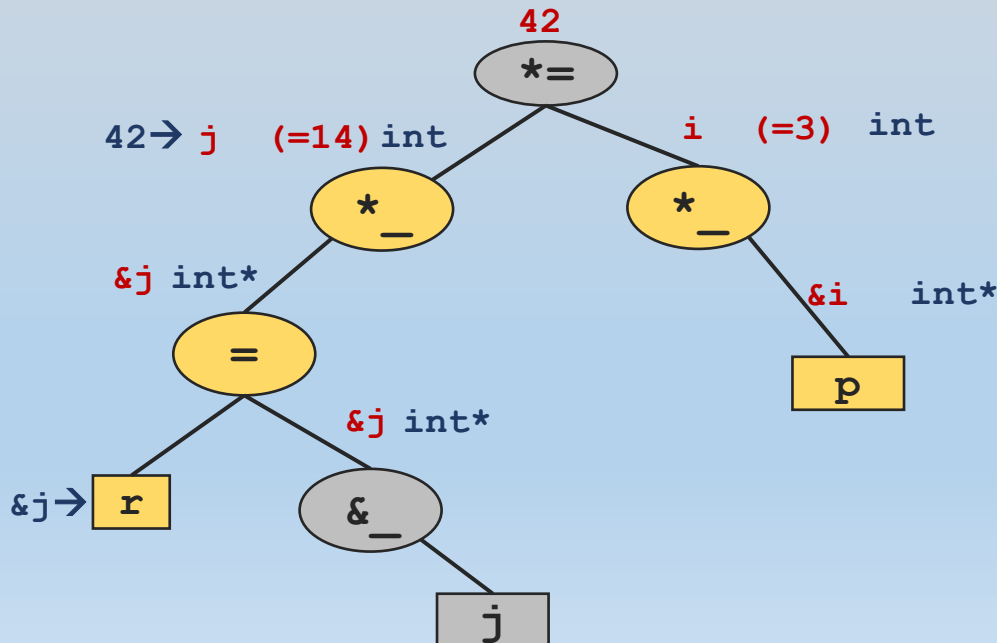
This line of code is put into an expression tree

Example expression tree

int
int*

Ref and deref operators can be used in complex expressions

```
* (r=&j) *= *p;    /** same as r=&j; j*=i;    **/
```



Operator precedence :

- Access and Grouping: `()` (function call) `[]`, `.`, `->`
- Unary: `++`, `--`, `!` `()` (type casting), `&` (address), `*` (dereference)
- Arithmetic: `*` (multiplication), `/`, `%`; and then `+`, `-`
- Bitwise shifts: `<<`, `>>`
- Relational: `>`, `>=`, `<`, `<=`
- Bitwise `&`; and then `^`; and then bitwise `|`
- Logical `&&`; and then logical `||`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`, etc.

Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7 (==14)      **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;          **/
p=&(i=3);          /** compilation error            **/
  
```

- You can only **reference a variable**
 error: invalid lvalue in unary '&'
- Error is that can't ref if not a variable
 - Same error for: `p=&(i+3);`

Referencing / dereferencing

Code example:

```

int      i = 3, j = 5,
        *p = &i, *q = &j, *r;

if(p == &i) printf ("same address\n");
j=**&p;           /** same as j=*p (==i)          **/
j = 7**p / *q+7;  /** same as j=7*i/j+7(==14)      **/
*(r=&j) *= *p;     /** same as r=&j; j*=i;          **/
p=&(i=3);          /** compilation error            **/
*i;               /** compilation error            **/
  
```

- You can **dereference any expression** that is of pointer type, but **only pointer types**
 error: invalid type argument of 'unary *'

Referencing / dereferencing - summary

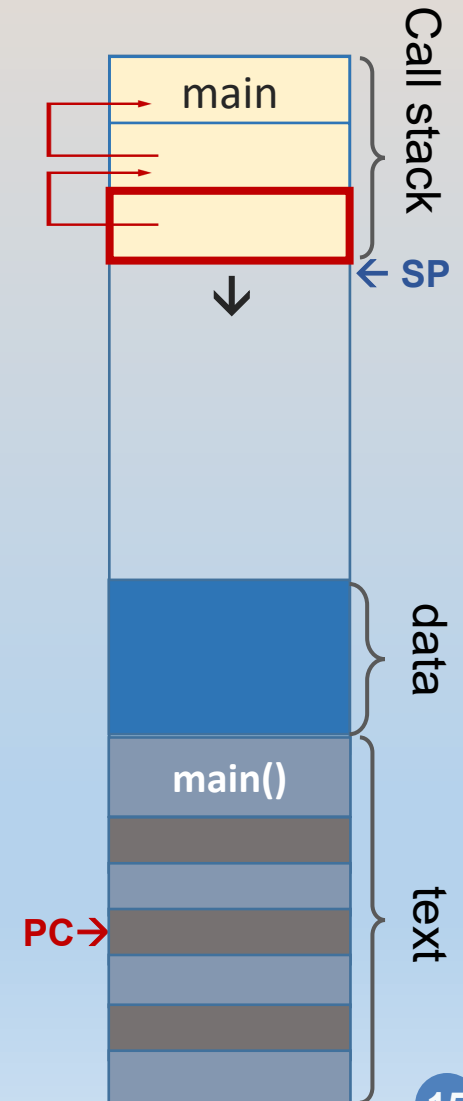
- Referencing (&) and dereferencing (*) are inverse operations
- The referencing operator (&) can be applied only to variables, but of any type (even a pointer)
- The dereferencing operator (*) can be applied to any expression as long as it is of pointer type
- If dereferencing is done on the right side of an assignment, then the content can be read
- If dereferencing is done on the left side of an assignment, then the content of that address can be modified

Lecture overview

- Pointers in C – recap
- Pointers as function parameters ←
- Arrays in C
- Arrays and pointers
- Strings in C
- Pointer arithmetic and array indexing

Memory segments

- Memory structure of a program is divided among several segments – text, data, stack
- Local variables are stored in **stack frames** of their invoked functions
- At each point, the program has access only to local variables in the “top” stack frame (as well as global vars and local static vars)
- With pointers we can gain access to local vars in other stack frames and bypass the “scope borders”



Pointers as function parameters

Using pointers as function parameters allows you to give a function access to other functions' stack frames

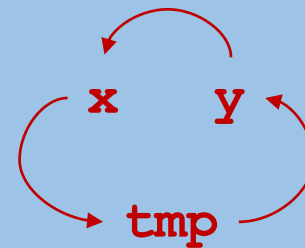
This is useful for various reasons:

- To implement call by reference ←
- To return more than one value
- To pass “by reference” large data structures

Call by value

- Recall that parameter passing in C is always done **by value** (the value of an expression is copied to the parameter)

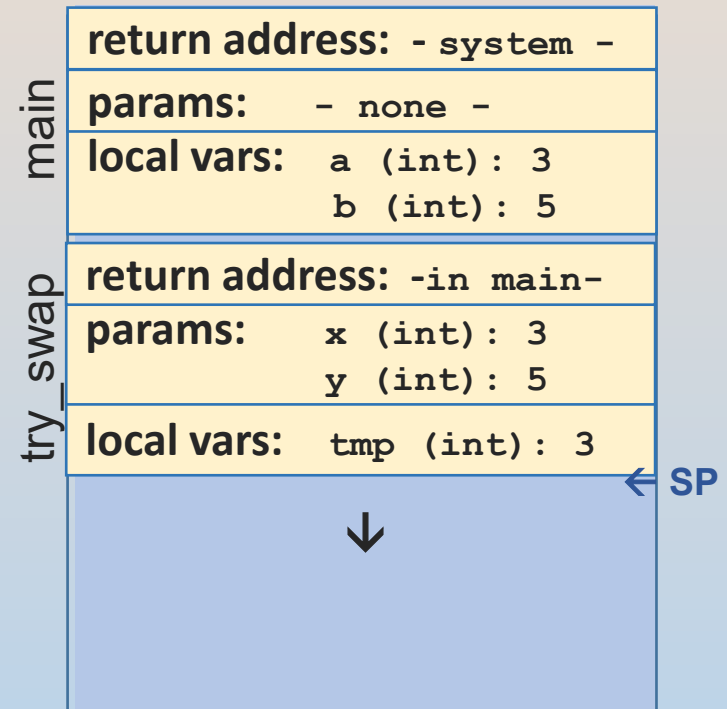
```
void try_swap (int x, int y) {  
    int tmp = x;  
    x = y;  
    y = tmp;  
}  
  
int main() {  
    int a=3,b=5;  
    printf("a=%d, b=%d\n", a, b);  
    try_swap(a,b);  
    printf("a=%d, b=%d\n", a, b);  
    return 0;  
}
```



Call by value

In the call stack:

- The values of local vars **a**, **b** in **main()** are copied to params **x**, **y** in **try_swap()**



PC →

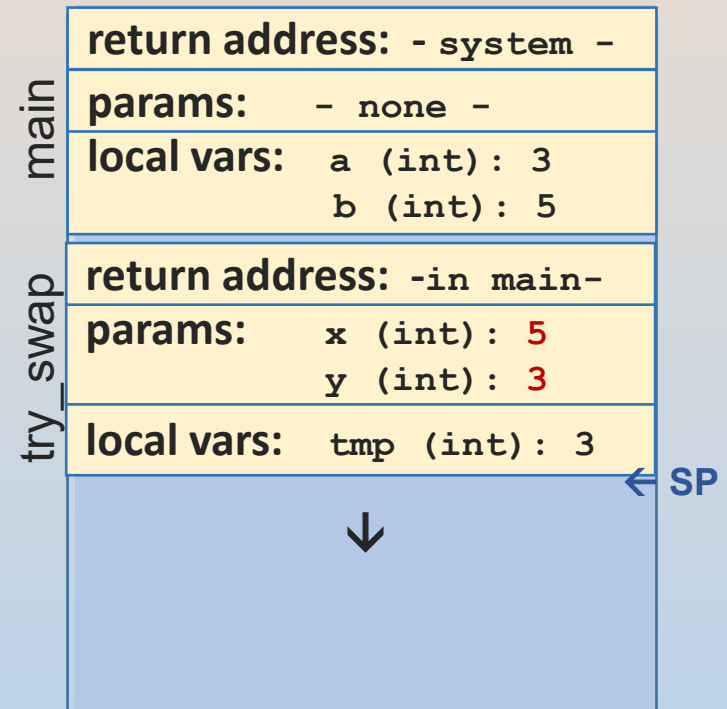
```

try_swap:
    tmp = x;
    x = y;
    y = tmp;
    return;
        
```

Call by value

In the call stack:

- The values of local vars **a**, **b** in `main()` are copied to params **x**, **y** in `try_swap()`
- When `try_swap()` returns, the values of params **x**, **y** are swapped, but values of **a**, **b** are unaffected
- This is because `try_swap()` does not have access to `main`'s stack frame



```

try_swap:
    tmp = x;
    x = y;
    y = tmp;
PC →  return;
  
```

Call by value

- Recall that parameter passing in C is always done **by value** (the value of an expression is copied to the parameter)

```
void try_swap (int x, int y) {  
    int tmp = x;  
    x = y;  
    y = tmp;  
}  
  
int main() {  
    int a=3,b=5;  
    printf("a=%d, b=%d\n", a, b);  
    try_swap(a,b);  
    printf("a=%d, b=%d\n", a, b);  
    return 0;  
}
```

```
a=3, b=5  
a=3, b=5
```

Swapping failed!

Call by reference

- However, a variable might be passed **by reference**, if its address is passed as argument to the function

```
void swap (int *p_x, int *p_y) {  
    int tmp = *p_x;  
    *p_x = *p_y;  
    *p_y = tmp;  
}
```

←function parameters
are pointer types

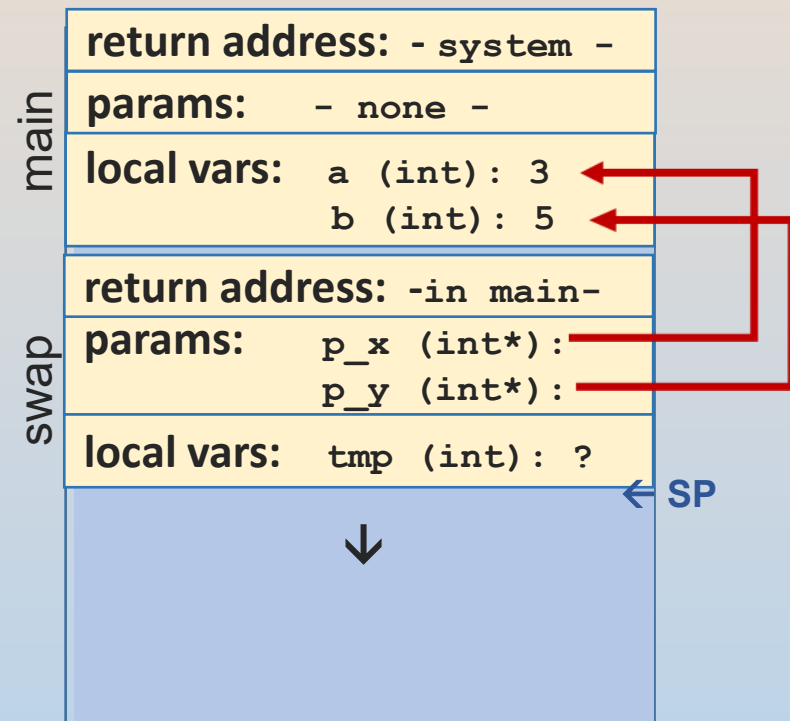
```
int main() {  
    int a=3,b=5;  
    printf("a=%d, b=%d\n", a, b);  
    swap(&a, &b);  
    printf("a=%d, b=%d\n", a, b);  
    return 0;  
}
```

←pass address of **a** and **b**

Call by reference

In the call stack:

- The **addresses** of local vars `a, b` in `main()` are copied to params `p_x, p_y` in `swap()`
- This gives `swap()` access to the local vars of `main()`



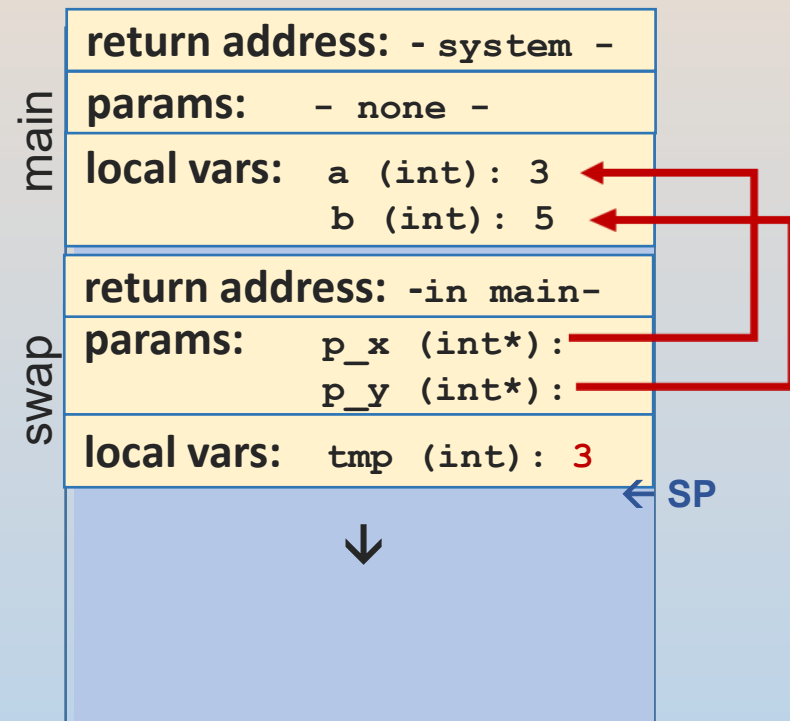
```

swap:
PC → tmp = *p_x;
      *p_x = *p_y;
      *p_y = tmp;
      return;
  
```

Call by reference

In the call stack:

- The **addresses** of local vars `a, b` in `main()` are copied to params `p_x, p_y` in `swap()`
- This gives `swap()` access to the local vars of `main()`



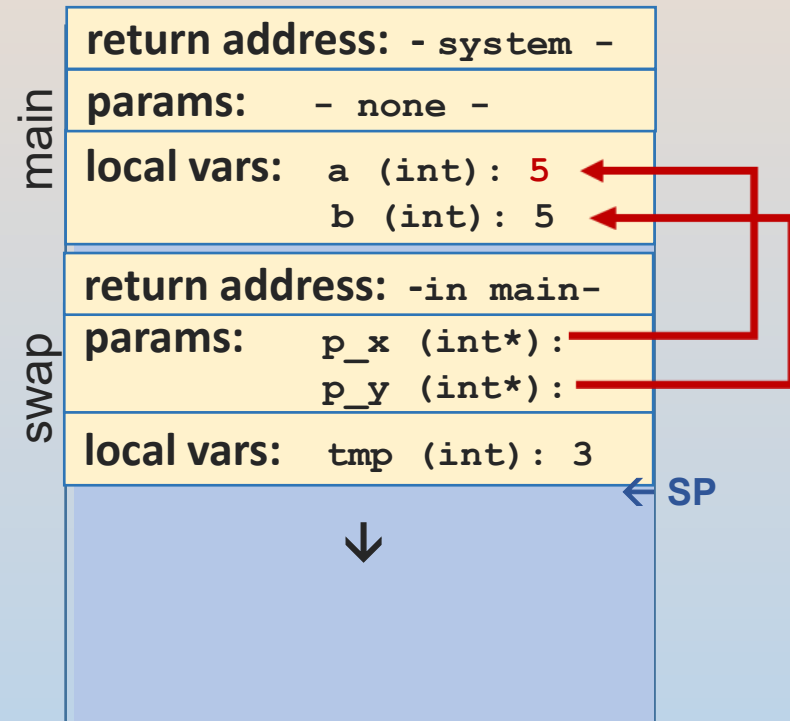
```

swap:
    tmp = *p_x;
    PC → *p_x = *p_y;
    *p_y = tmp;
    return;
  
```

Call by reference

In the call stack:

- The **addresses** of local vars `a, b` in `main()` are copied to params `p_x, p_y` in `swap()`
- This gives `swap()` access to the local vars of `main()`



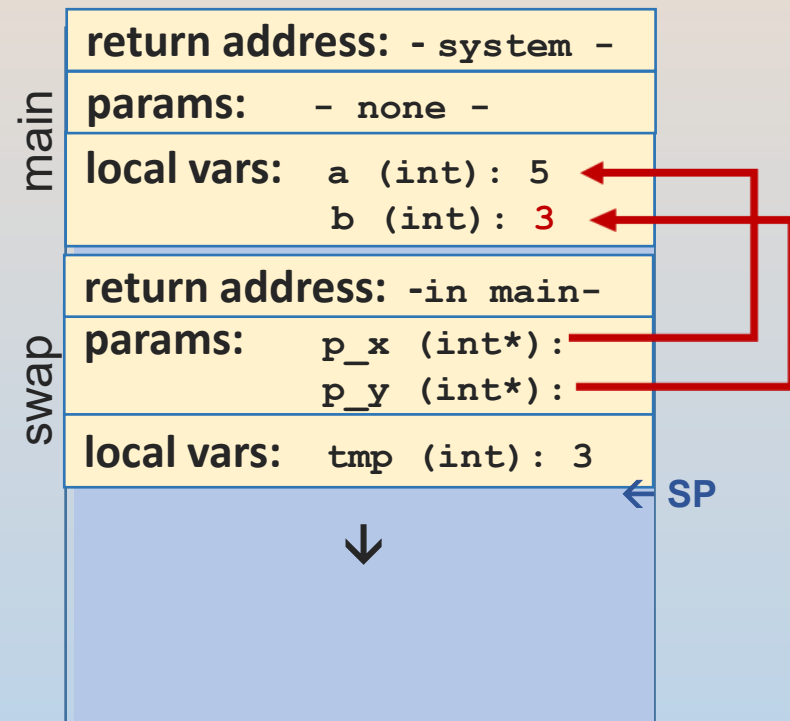
```

swap:
    tmp = *p_x;
    *p_x = *p_y;
    PC → *p_y = tmp;
    return;
  
```


Call by reference

In the call stack:

- The **addresses** of local vars `a, b` in `main()` are copied to params `p_x, p_y` in `swap()`
- This gives `swap()` access to the local vars of `main()`



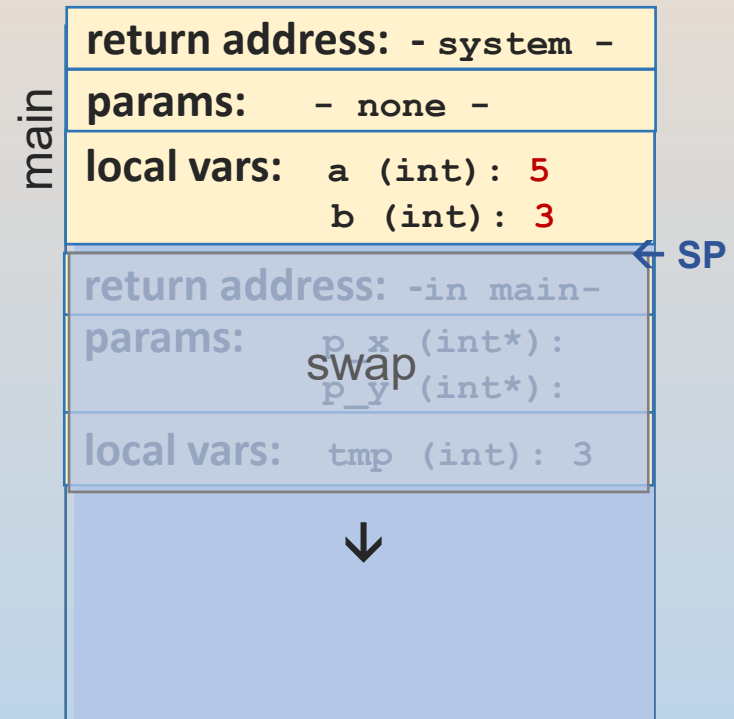
```

swap:
    tmp = *p_x;
    *p_x = *p_y;
    *p_y = tmp;
    PC → return;
  
```

Call by reference

In the call stack:

- The **addresses** of local vars **a**, **b** in **main()** are copied to params **p_x**, **p_y** in **swap()**
- This gives **swap()** access to the local vars of **main()**
- When **swap()** returns, the values of **main's** local vars **a**, **b** are swapped



Pointers as function parameters

Using pointers as function parameters allows you to give a function access to other functions' stack frames

This is useful for various reasons:

- To implement call by reference
- To return more than one value ←
- To pass “by reference” large data structures

Returning multiple values

- A function can return only a single value
- If your function needs to return multiple values, give it a list of addresses where the return values will be written

Returning multiple values

- A function can return only a single value
- If your function needs to return multiple values, give it a list of addresses where the return values will be written
- Example 1:

– `scanf()`

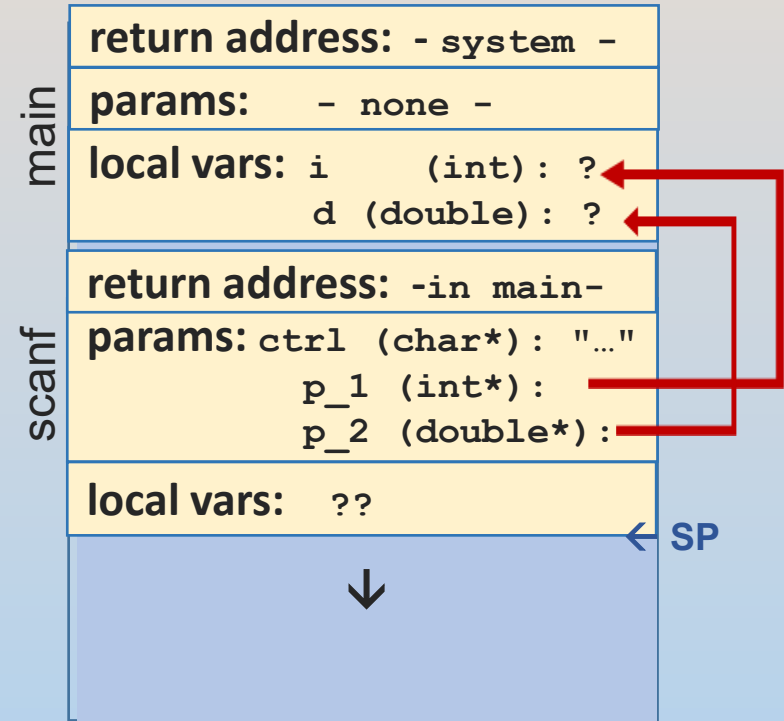
```
int i;  
double d;  
  
scanf("%d %lf",&i, &d);
```

- `scanf` receives a control string and a list of references to variables, where text from `stdin` should be copied to

Returning multiple values

In the call stack:

- When scanf() is called, it is passed pointers to variables in the calling function
- scanf() updates the values of these variables with values entered in stdin



Returning multiple values

- A function can return only a single value
- If your function needs to return multiple values, give it a list of addresses where the return values will be written
- Example 2:
 - possible function for integer division:

```
int enum=45, denom=7, quotient, remainder;  
divide(enum, denom, &quotient, &remainder);  
printf("%d/%d = %d + %d/%d\n",  
       enum, denom, quotient, remainder, denom);
```

← in main ()

45/7 = 6 + 3/7

- **divide** receives two integers and the addresses of two int variables where the quotient and remainder will be written

Returning multiple values

- A function can return only a single value
- If your function needs to return multiple values, give it a list of addresses where the return values will be written
- Example 2:
 - possible function for integer division:

```
int enum=45, denom=7, quotient, remainder;  
divide(enum, denom, &quotient, &remainder);  
printf("%d/%d = %d + %d/%d\n",  
        enum, denom, quotient, remainder, denom);
```

← in main ()

note void → return type

```
void divide (int x, int y, int *p_quot, int *p_remain) {  
    *p_quot    = x / y;  
    *p_remain  = x % y;  
    return;  
}
```

- parameters **x, y** are regular 'input' parameters
- parameters **p_quot, p_remain** are 'output' parameters

Pointers as function parameters

Using pointers as function parameters allows you to give a function access to other functions' stack frames

This is useful for various reasons:

- To implement call by reference
- To return more than one value
- To pass “by reference” large data structures ←
such as **arrays** and other “objects”
 - We discuss **arrays** today
 - We discuss general objects and other complex data structures later in the course

Online poll

Answer the questions considering these variable definitions:

```
int j;  
int *p=&j, i;  
double d1=2.5, d2, *q=&d2;
```

1. What is the type of variable **i**?
 - `int`
 - `int*` (pointer to `int`)
2. What is the type of variable **p**?
 - `int`
 - `int*` (pointer to `int`)
3. After the expression `d2 = *(q=&d1)` we have: *(multiple answers possible)*
 - **q** contains the address of **d1**
 - **q** contains the address of **d2**
 - **d1** and **d2** contain the value 2.5
 - Syntax error
4. Which of these expression results in compilation warnings/errors: *(multiple answers possible)*
 - `*p = i+3`
 - `*(p=&i)+3`
 - `*p=&i`
 - `d1=(*q)++`
 - `d2=*p`
 - `q=*i`

Online poll – solutions

1. What is the type of variable `i`?

- `int` ←
- `int*` (pointer to `int`)

2. What is the type of variable `p`?

- `int`
- `int*` (pointer to `int`) ←

```
int j;  
int *p=&j, i;  
double d1=2.5, d2, *q=&d2;
```

Online poll – solutions

3. After the expression `d2 = *(q=&d1)` we have:

- `q` contains the address of `d1` ←
- `q` contains the address of `d2`
- `d1` and `d2` contain the value 2.5 ←
- Syntax error

```
int j;  
int *p=&j, i;  
double d1=2.5, d2, *q=&d2;
```

Online poll – solutions

Operator precedence :

- Access and Grouping: `()` (function call), `[]`, `.`, `->`
- Unary: `++`, `--`, `!`, `()` (type casting), `&` (address), `*` (dereference)
- Arithmetic: `*` (multiplication), `/`, `%`; and then `+`, `-`
- Bitwise shifts: `<<`, `>>`
- Relational: `>`, `>=`, `<`, `<=`
- Bitwise `&`; and then `^`; and then bitwise `|`
- Logical `&&`; and then logical `||`
- Assignment: `=`, `+=`, `-=`, `*=`, `/=`, etc.

4. Which of these expression results in compilation warnings/errors:

- `*p = i+3` [value `i+3` is copied to address written in `p`]
- `*(p=&i)+3` [`p` points to `i` ; expression value is `i+3`]
- `*p=&i` [`p` is type `int*` so `*p` (left side) is type `int`; `&i` (right side) is type `int*`]
- `d1=(*q)++` [the value pointed to by `q` is copied to `d1` and incremented]
- `d2=*p` [the value pointed to by `p` is converted into double and copied to `d2`]
- `q=*i` [cannot de-reference a value (`i`) of type `int`]

```
int j;  
int *p=&j, i;  
double d1=2.5, d2, *q=&d2;
```

Lecture overview

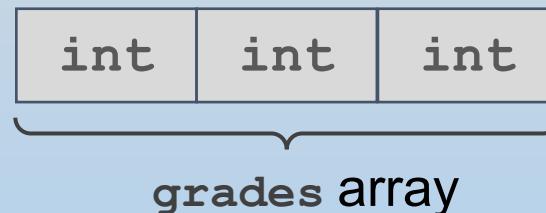
- Pointers in C – recap
- Pointers as function parameters
- **Arrays in C ←**
- Arrays and pointers
- Strings in C
- Pointer arithmetic and array indexing

Arrays

An array is a sequence of variables of the same type allocated contiguously in memory

- Definition of an array of three `int` variables:

```
int grades[3];
```



The size of an array defined as a variable **must be constant** (known at compile time)

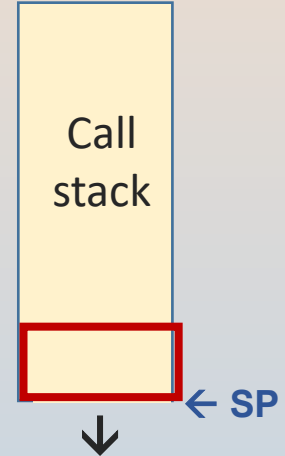
Unlike Java!

- ➔ Non constant array size will **not** trigger a compilation error or warning, but only works for automatic variables and only on some C compilers (C89: no, C99: yes, C11: optional!)

Array storage

The arrays we will discuss here are hard-coded in the program:

- **Global** (**extern**) arrays are allocated in the data segment
- **Local** (**auto**) are allocated in the stack frame of the function they are defined in

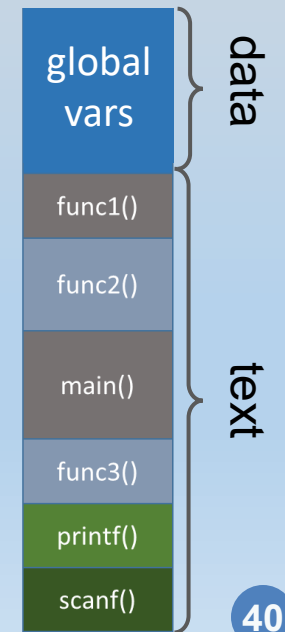


The length of the array cannot be determined at run-time!

Unlike Java!

Arrays of dynamic size (determined at run-time) can be allocated on the heap [we will discuss next week]

In C, whenever possible, we use constant-size arrays



Array initialization

Array entries can be initialized manually as follows:

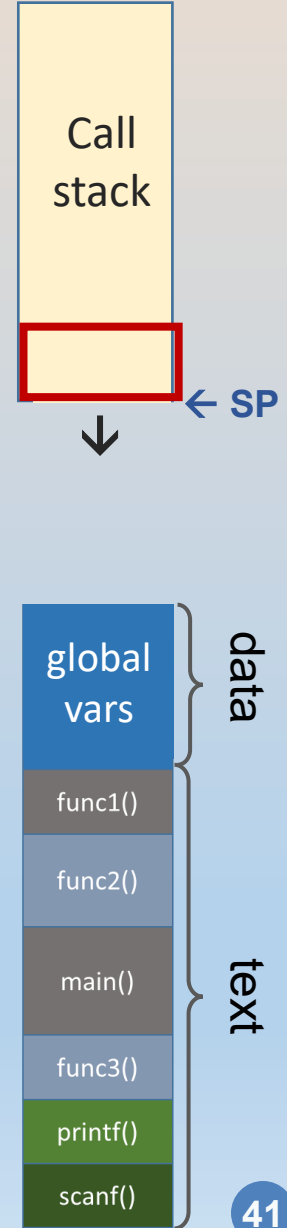
```
float array1[5] = { 0.0, 1.0, 2.0, 3.0, 4.0 };
```

Alternative initializations:

```
char array2[] = { 'a', 'b', 'c' }; ← no need to specify array size if
initialization is same time as declaration
```

```
float array3[5]; ← size specified; no entries specified
float array4[5] = { 1, 2, 3 }; ← size specified; some entries specified
```

Does the compiler automatically initialize arrays?



Array initialization

Array entries can be initialized manually as follows:

```
float array1[5] = { 0.0, 1.0, 2.0, 3.0, 4.0 };
```

Alternative initializations:

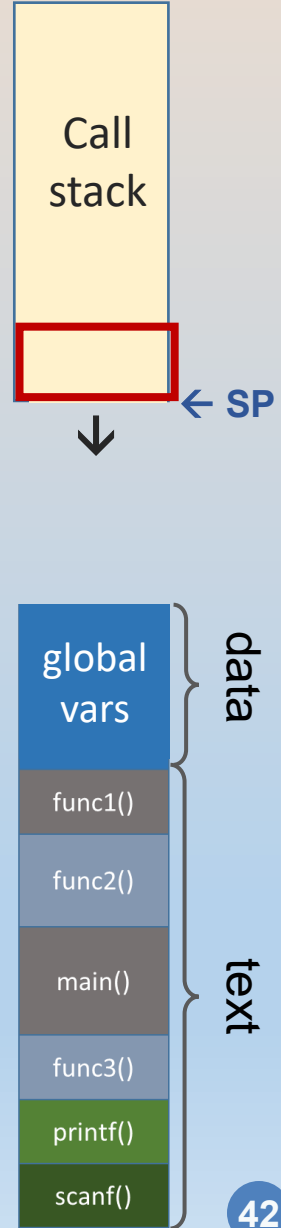
```
char array2[] = { 'a', 'b', 'c' }; ← no need to specify array size if
initialization is same time as declaration
```

```
float array3[5]; ← size specified; no entries specified
```

```
float array4[5] = { 1, 2, 3 }; ← size specified; some entries specified
```

Does the compiler automatically initialize arrays?

- **Global** (**extern**) arrays are always initialized by setting all of their elements to zero by default
- **Local** (**auto**) are not automatically initialized and thus they should be assumed to contain garbage
 - Unless: if some entries specified then remaining entries set to 0



Array allocation example

```

char word1[10];

int main() {
    char word2[5];
    char word3[5]={1};
    ...
    return 0;
}
    
```

main

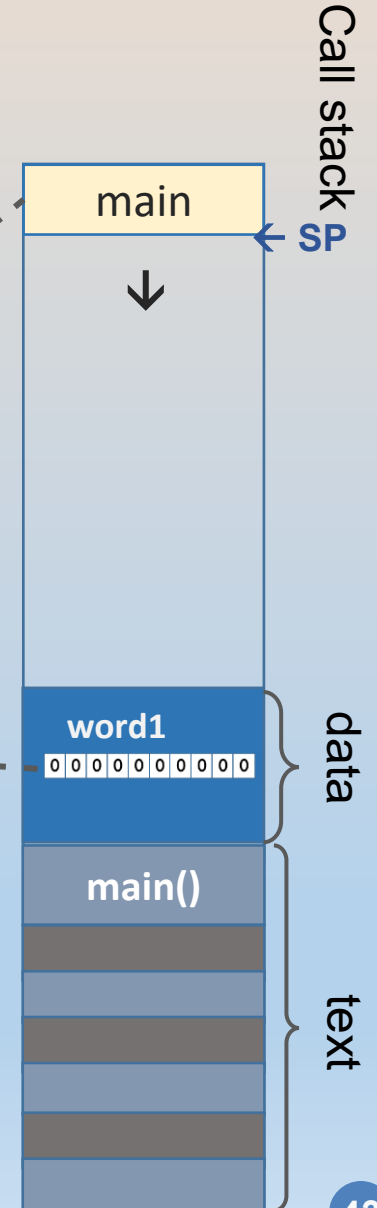
return address: - system -
params: - none -
local vars: word2 (char*)
? ? ? ? ?
word3 (char*)
1 0 0 0 0

0 0 0 0 0 0 0 0 0 0

Does the compiler automatically initialize arrays?

Example:

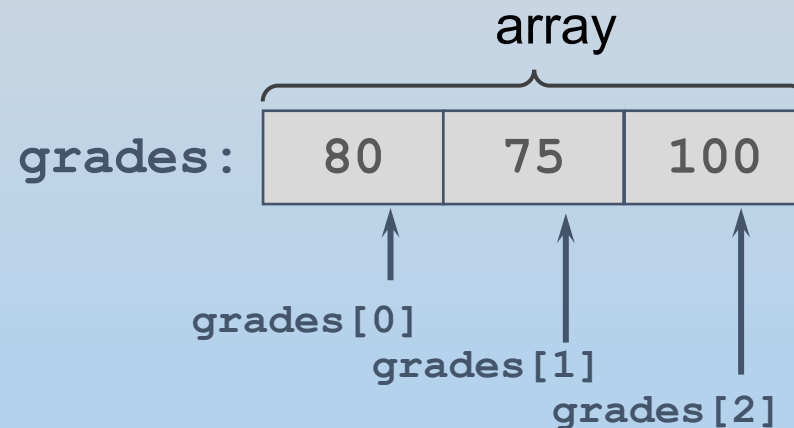
Global (**extern**) arrays vs. **Local** (**auto**) arrays



Accessing array entries

Array entries can be accessed using the `[]` operator:

```
int grades[3] = {80, 75, 100};  
int a = grades[2]; // a = 100  
grades[1]++; // grades[1] = 76
```



Arrays – example

```
#include <stdio.h>

int main() {
    int i, a[5] = {12, 32, 434};

    for (i = 0; i < 5; i++)
        printf ("%5d", a[i]);
    printf ("\n");
    return 0;
}
```

12	32	434	0	0
----	----	-----	---	---

Arrays – example

```
#include <stdio.h>
#define SIZE 5

int main() {
    int i, a[SIZE] = {12, 32, 434};

    for (i = 0; i < SIZE; i++)
        printf ("%5d", a[i]);
    printf ("\n");
    return 0;
}
```

12	32	434	0	0
----	----	-----	---	---

- Arrays sizes are commonly defined using **symbolic constants**
- Symbolic constants are defined using **#define directives**
- **#define** directives are used to specify **text substitutions** in C source code

Arrays – example

```
#include <stdio.h>
#define SIZE 5

int main() {
    int i, a[SIZE] = {12, 32, 434};

    for (i = -1; i <= SIZE; i++)
        printf ("%5d", a[i]);
    printf ("\n");
    return 0;
}
```

-1 12 32 434 0 0 32766

Unlike Java!

- C does not contain a mechanism to guard from **overflowing** from array boundaries (even negative indices are allowed)
- We discuss the mechanism behind array indexing later today ...

Lecture overview

- Pointers in C – recap
- Pointers as function parameters
- Arrays in C
- **Arrays and pointers ←**
- Strings in C
- Pointer arithmetic and array indexing

Arrays and pointers

- A variable associated with an array actually holds a pointer to the first element in the array
- This address can be copied to another pointer variable and the array can be accessed through that variable

Example:

```
int i, *p, a[SIZE] = {12, 32, 434};  
p = a;  
for (i = 0; i < SIZE; i++)  
    printf ("%5d", p[i]);
```



- Both **a** and **p** are of type **int***
- There are subtle differences between the two, which we will discuss next week

Passing arrays as function arguments

This is very useful when passing arrays to functions as parameters:

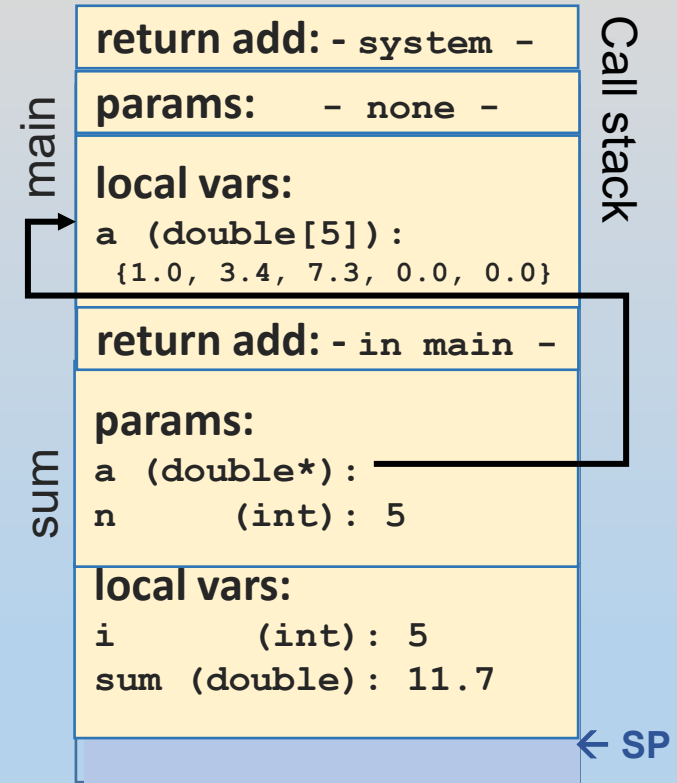
```

#define SIZE 5

double sum (double*, int);

int main() {
    double a[SIZE] = {1.0, 3.4, 7.3};
    printf("sum of array is %lf\n", sum(a, SIZE));
    return 0;
}

double sum (double *a, int n) {
    int i;
    double sum = 0.0;
    for (i = 0; i < n; ++i)
        sum += a[i];
    return sum;
}
    
```



Passing arrays as function arguments

This is very useful when passing arrays to functions as parameters:

```
#define SIZE 5

double sum (double*, int);

int main() {
    double a[SIZE] = {1.0, 3.4, 7.3};
    printf("sum of array is %lf\n", sum(a, SIZE));
    return 0;
}

double sum (double a[], int n) {← same as double *a
    int i;
    double sum = 0.0;
    for (i = 0; i < n; ++i)
        sum += a[i];
    return sum;
}
```

- Notation [] can also be used to define a pointer parameter

Passing arrays as function arguments

This is very useful when passing arrays to functions as parameters:

```
#define SIZE 5

double sum (const double*, int);

int main() {
    double a[SIZE] = {1.0, 3.4, 7.3};
    printf("sum of array is %lf\n", sum(a,SIZE));
    return 0;
}

double sum (const double a[], int n) {
    int i;
    double sum = 0.0;
    for (i = 0; i < n; ++i)
        sum += a[i];
    return sum;
}
```

- Notation **const** before pointer type implies that you're not writing into the array (**function has read only access**)

Pointers as function parameters

Using pointers as function parameters allows you to give a function access to other functions' stack frames

This is useful for various reasons:

- To implement call by reference
- To return more than one value
- To pass “by reference” large data structures ←
such as **arrays** and other “objects”
 - We discuss **arrays** today
 - We discuss general objects and other complex data structures later in the course

Online poll

Answer the questions considering the following code:

```
#define SIZE 10

int sum (int*, int);

int main() {
    int array[SIZE] = {2, 0, 3, 5};
    printf("sum of array is %d\n",
           sum(array, 6));
    return 0;
}

int sum (int a[], int n) {
    int i;
    int sum = 0;
    for (i = 0; i < n; ++i)
        sum += a[i];
    return sum;
}
```

1. Array **array**[] is allocated:
 - on the stack
 - in the data segment
 - in the text segment
2. Array **array**[] contains:
 - four elements
 - ten elements; 5th-10th elements contain garbage values
 - ten elements; 5th-10th elements contain 0 values
3. The stack frame of function **sum()**:
 - contains a complete copy of **array**[] (all its elements)
 - contains a partial copy of **array**[] (some of its elements)
 - contains a pointer to **array**[]
4. See poll...

Online poll – solutions

1. Array `array[]` is allocated:

- on the stack ←
- in the data segment
- in the text segment

```
#define SIZE 10
int sum (int*, int);
int main() {
    int array[SIZE] = {2, 0, 3, 5};
    printf("sum of array is %d\n", sum(array,6));
    return 0;
}

int sum (int a[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; ++i) { sum += a[i]; }
    return sum;
}
```

Online poll – solutions

2. Array `array[]` contains:

- four elements
- ten elements; 5th-10th elements contain garbage values
- ten elements; 5th-10th elements contain 0 values ←

```
#define SIZE 10
int sum (int*, int);
int main() {
    int array[SIZE] = {2, 0, 3, 5};
    printf("sum of array is %d\n", sum(array,6));
    return 0;
}

int sum (int a[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; ++i) { sum += a[i]; }
    return sum;
}
```


Online poll – solutions

3. The stack frame of function `sum()` contains:
- a complete copy of `array[]` (all its elements)
 - a partial copy of `array[]` (some of its elements)
 - a pointer to `array[]` ←

```
#define SIZE 10
int sum (int*, int);
int main() {
    int array[SIZE] = {2, 0, 3, 5};
    printf("sum of array is %d\n", sum(array,6));
    return 0;
}

int sum (int a[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; ++i) { sum += a[i]; }
    return sum;
}
```

Online poll – solutions

4. Changing the call to function `sum` to `sum(array, 20)` will result in:

- a compilation error or warning
- a runtime error and program crash
- an unexpected return value (not necessarily 10) ←
- The same return value (10)

```
#define SIZE 10
int sum (int*, int);
int main() {
    int array[SIZE] = {2, 0, 3, 5};
    printf("sum of array is %d\n", sum(array,6));
    return 0;
}

int sum (int a[], int n) {
    int i, sum = 0;
    for (i = 0; i < n; ++i) { sum += a[i]; }
    return sum;
}
```

Lecture overview

- Pointers in C – recap
- Pointers as function parameters
- Arrays in C
- Arrays and pointers
- **Strings in C ←**
- Pointer arithmetic and array indexing

Strings in C

- Strings in C are implemented using **character arrays**
 - The array of an n -character string has $n+1$ entries, where last element is set to '**\0**' which has the value of zero in ASCII
 - The terminating '**\0**' is not a printable character, so it provides a way for **encoding the string boundary**

- Simple is to use string literal to initialize array:

```
char s[] = "abc";
```

- Alternatively, define string "abc" as regular array (same consequence):

```
char s[] = {'a', 'b', 'c', '\0'}; // if '\0' not specified, then  
// array has 3 entries and no good way to encode string boundary
```

- Using string literal with free pointer
(string allocated in data segment with read-only access):

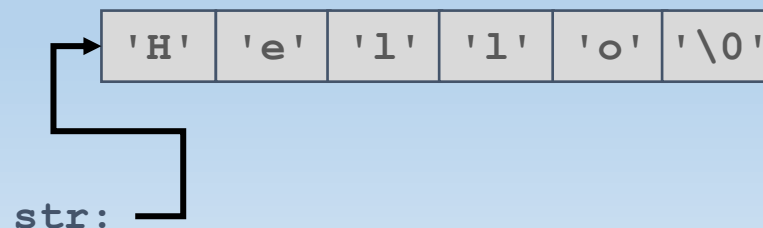
```
char *s = "abc";
```

Strings functions

A function for computing the **length** of a string:

```
int my_strlen(const char* str) {  
    int len=0;  
    while(str[len++]) // shorthand for: while(str[len++] != 0)  
        // since ASCII 0 = '\0' then: while(str[len++] != '\0')  
        ;  
    return len-1;  
}
```

- The string is located in a given character array (buffer)
- The function does not know where this array ends
- It computes the number of chars until the first ' \0 '



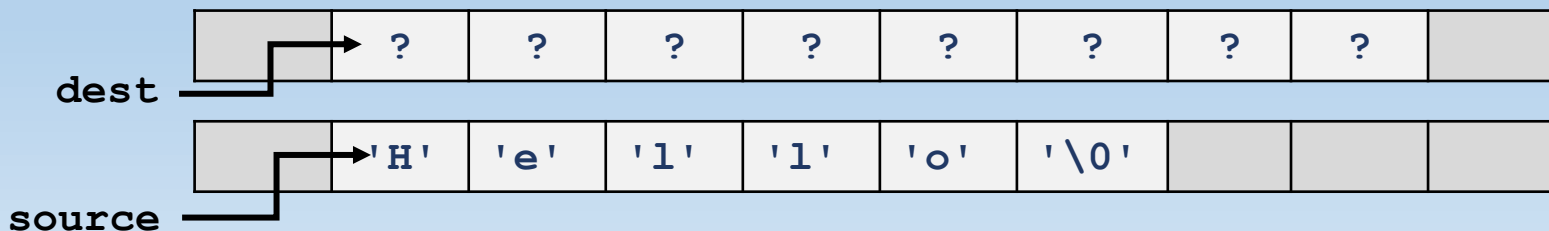
Strings functions

A function for **copying** a string into a given array (buffer):

```

char* my_strcpy(char* dest, const char* source) {
    int i=0;
    while(source[i]) {
        dest[i] = source[i];
        i++;
    }
    dest[i] = '\0';
    return dest;
}
    
```

- The calling function provides two arrays, one containing a string (**source**) and the other one (**dest**) is a **buffer** with sufficient space



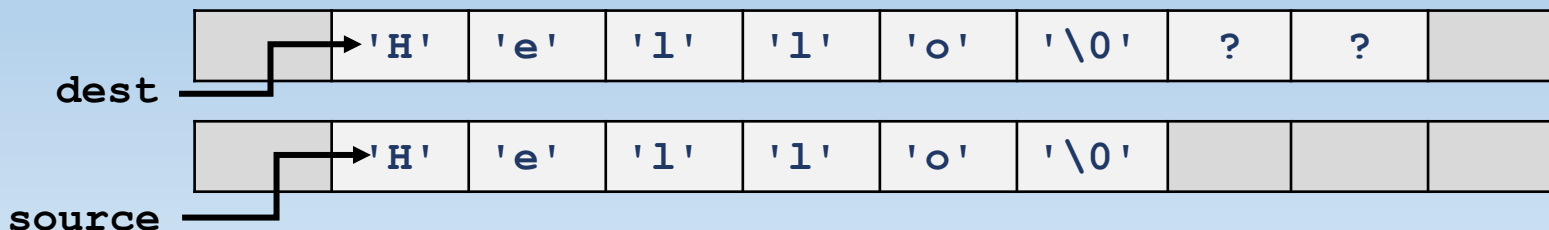
Strings functions

A function for **copying** a string into a given array (buffer):

```

char* my_strcpy(char* dest, const char* source) {
    int i=0;
    while(source[i]) {
        dest[i] = source[i];
        i++;
    }
    dest[i] = '\0';
    return dest;
}
    
```

- The calling function provides two arrays, one containing a string (**source**) and the other one (**dest**) is a **buffer** with sufficient space
- The source string is copied to the destination buffer



Strings functions

A function for **copying** a string into a given array (buffer):

```
char* my_strcpy(char* dest, const char* source) {  
    int i=0;  
    while(source[i]) {  
        dest[i] = source[i];  
        i++;  
    }  
    dest[i] = '\\0';  
    return dest;  
}
```

- The calling function provides two arrays, one containing a string (**source**) and the other one (**dest**) is a **buffer** with sufficient space
- The source string is copied to the destination buffer
- The function returns a pointer to the destination string
- If the destination buffer doesn't contain enough cells, it may overwrite memory cells that belong to other variables

Printing strings

A string can be printed to `stdout` using `printf` with the `%s` conversion specifier

```
#include <stdio.h>

int main() {
    char *str="Hello";
    printf("str = %s\n", str);

    return 0;
}
```

```
str = Hello
```

Strings functions

- Standard library `string.h` contains many functions that work with strings:
 - `strlen()` – string length
 - `strcpy()` – copies string
 - `strncpy()` – copies string, up to given num chars
 - `strcat()` – concats string
 - `strncat()` – concats string, up to given num chars
 - `strcmp()` – lexicographic comparison of strings (returns < 0 , 0 , or > 0)
 - `strchr()` – returns pointer to first occurrence of given char in string
- Many more...
- See <http://www.cplusplus.com/reference/cstring/>

Lecture overview

- Pointers in C – recap
- Pointers as function parameters
- Arrays in C
- Arrays and pointers
- Strings in C
- Pointer arithmetic and array indexing ←

Accessing array entries via pointer arithmetic

```
int grades[3];
```

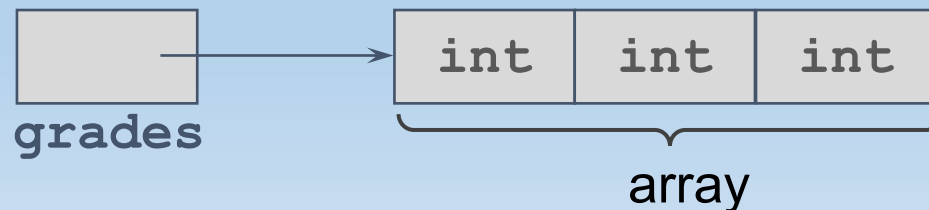
- Variable **grades** (type `int*`) holds address of first byte of array
- 1st entry (index 0) can be accessed by:

```
int num = grades[0];
```

or

```
int num = *grades;
```

(de-referencing **grades** returns the `int` value it points to)



Accessing array entries via pointer arithmetic

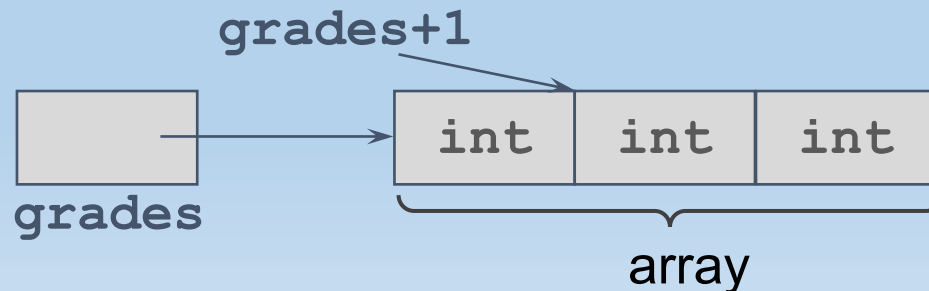
```
int grades[3];
```

- Variable **grades** (type `int*`) holds address of first byte of array
- The value of **grades+1** is the address of **grades[1]**

```
int num = grades[1];
```

or

```
int num = *(grades+1);
```



Accessing array entries via pointer arithmetic

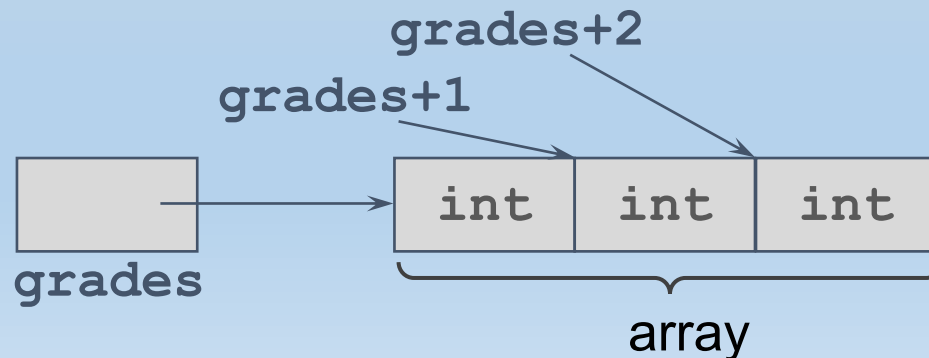
```
int grades[3];
```

- Variable **grades** (type **int***) holds address of first byte of array
- And in general, **grades+i** is the address of **grades[i]**

```
int num = grades[i];
```

or

```
int num = *(grades+i);
```



Pointer arithmetic operations

- `ptr` is an address of a certain pointer **type** (e.g. , `int*` or `double*`)
- `*ptr` is some integer (positive or negative)
- `ptr + i` is the address held by `ptr + ( i x sizeof(type) )`

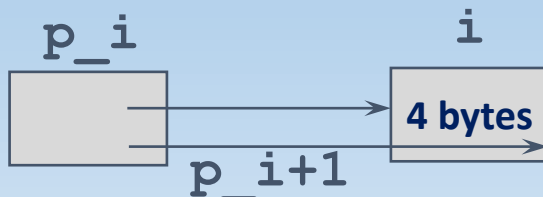
Example:

```

int    i = 7,
      *p_i = &i;

printf("p_i=%p, p_i+1=%p\n", p_i, p_i+1);
    
```

`p_i=0xbfffffff4d4`, `p_i+1=0xbfffffff4d8` ←address difference of 4



Pointer arithmetic operations

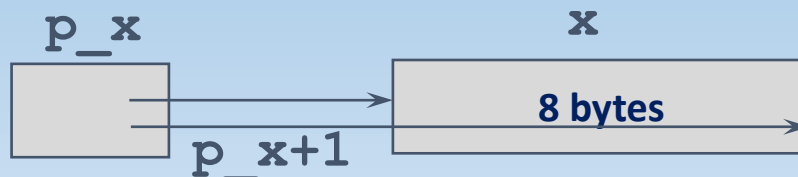
- `ptr` is an address of a certain pointer **type** (e.g. , `int*` or `double*`)
- `*ptr` is some integer (positive or negative)
- `ptr + i` is the address held by `ptr + ( i x sizeof(type) )`

Example:

```
double x = 7.0,
      *p_x = &x;

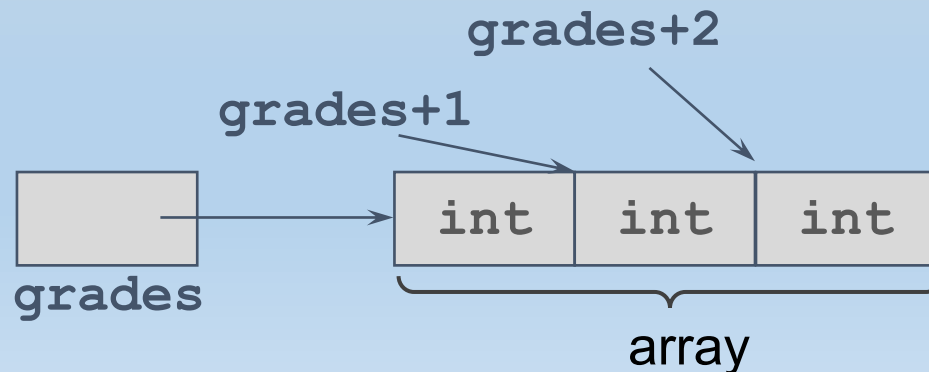
printf("p_x=%p, p_x+1=%p\n", p_x, p_x+1);
```

`p_x=0xbfffffff4c0`, `p_x+1=0xbfffffff4c8` ←address difference of 8



Pointer arithmetic operations

- `ptr` is an address of a certain pointer **type** (e.g. , `int*` or `double*`)
- `ptr + i` is the address held by `ptr + ( i x sizeof(type) )`
- The notation `ptr[i]` is simply shorthand for `*(ptr+i)`



Arrays and pointers – summary

- Array variables are of pointer type
- Array indexing is implemented via pointer arithmetic
$$\text{arr}[i] \equiv *(arr+i)$$
(one reason why pointer type matters. We'll see more next week)
- Free pointers are pointer variables not associated with an array definition
- Both array and free pointers can be de-referenced using `*` or `[]`
- Function parameters are always free pointers
(even if `[]` notation is used in type of parameter)
- There are two main differences between array variables and free pointers (to be continued next week . . .)

At home...

- Homework exercise #4 due **Sunday, May 31 @ 21:00**
- Homework exercise #5 will be published on Thursday
- Review class material on arrays, pointers, and strings.
We will discuss advanced topics next week . . .